SciPy Peak Finding Algorithms for SAXS/WAXS/GISAXS/GIWAXS

Source docs:
- SciPy signal peak finding reference:
https://docs.scipy.org/doc/scipy-1.17.0/reference/signal.html#peak-finding
- scipy.signal.find_peaks:
https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.signal.find_peaks.html
- scipy.signal.find_peaks_cwt:
https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.signal.find_peaks_cwt.html
- scipy.signal.peak_prominences:
https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.signal.peak_prominences.html
- scipy.signal.peak_widths:
https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.signal.peak_widths.html
- scipy.signal.argrelextrema:
https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.signal.argrelextrema.html
- scipy.signal.argrelmax:
https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.signal.argrelmax.html
- scipy.signal.argrelmin:
https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.signal.argrelmin.html

Use case: after detector images are integrated or otherwise reduced to a one-dimensional scattering curve, these algorithms identify Bragg peaks, broad scattering maxima, shoulders, and local extrema in intensity versus q, two-theta, pixel index, or retention coordinate.

1. scipy.signal.find_peaks

Functionality: finds local maxima in a 1D array and filters candidates by peak height, threshold, minimum distance, prominence, width, and plateau size.

Best use case: default peak detector for 1D SAXS/WAXS curves. Use prominence to reject baseline/noise features, distance to avoid duplicate nearby peaks, and width to reject features that are too narrow or too broad.

Snippet:
```python
import numpy as np
from scipy.signal import find_peaks, peak_widths, savgol_filter


def find_scattering_peaks(q, intensity):
    y = np.asarray(intensity, dtype=float)
    y = np.nan_to_num(y, nan=np.nanmedian(y))
    y_smooth = savgol_filter(y, window_length=11, polyorder=3)
    y_corrected = y_smooth - np.percentile(y_smooth, 10)

    peaks, props = find_peaks(
        y_corrected,
        prominence=0.05 * np.max(y_corrected),
        distance=8,
        width=2,
    )
    widths = peak_widths(y_corrected, peaks, rel_height=0.5)

    return [
        {
            "q": float(q[idx]),
            "intensity": float(y_corrected[idx]),
            "prominence": float(props["prominences"][i]),
            "fwhm_samples": float(widths[0][i]),
        }
        for i, idx in enumerate(peaks)
    ]
```

2. scipy.signal.peak_prominences

Functionality: measures how much each peak rises above its surrounding baseline. Prominence is

often more robust than raw height when scattering curves have sloping backgrounds.

Best use case: rank peaks after detection, remove weak shoulders, or compare peak quality across curves with different absolute intensity scales.

Snippet:
```python
import numpy as np
from scipy.signal import find_peaks, peak_prominences


def rank_peaks_by_prominence(q, intensity):
    y = np.asarray(intensity, dtype=float)
    peaks, _ = find_peaks(y, distance=8)
    prominences, left_bases, right_bases = peak_prominences(y, peaks)

    order = np.argsort(prominences)[::-1]
    return [
        {
            "q": float(q[peaks[i]]),
            "prominence": float(prominences[i]),
            "left_base_q": float(q[left_bases[i]]),
            "right_base_q": float(q[right_bases[i]]),
        }
        for i in order
    ]
```

3. scipy.signal.peak_widths

Functionality: estimates peak widths at a chosen relative height. With rel_height=0.5 this gives FWHM-like width in sample units.

Best use case: capture peak broadening, compare crystallinity/order, or prepare initial sigma guesses for later Gaussian/Lorentzian fitting.

Snippet:
```python
from scipy.signal import find_peaks, peak_widths


def measure_peak_widths(x, y):
    peaks, props = find_peaks(y, prominence=0.05 * max(y))
    widths, width_heights, left_ips, right_ips = peak_widths(y, peaks, rel_height=0.5)

    dx = x[1] - x[0]
    return [
        {
            "center_x": float(x[p]),
            "width_x": float(w * dx),
            "left_x": float(x[0] + left * dx),
            "right_x": float(x[0] + right * dx),
        }
        for p, w, left, right in zip(peaks, widths, left_ips, right_ips)
    ]
```

4. scipy.signal.find_peaks_cwt

Functionality: continuous wavelet transform peak finder. It searches across multiple widths, so it can detect broader peaks that simple local-maximum filtering may miss.

Best use case: noisy SAXS curves, broad amorphous halos, weak peaks where expected width range is known. More sensitive to width selection and preprocessing than find_peaks.

Snippet:
```python
```

```python
import numpy as np
from scipy.signal import find_peaks_cwt


def wavelet_peak_candidates(q, intensity, min_width=2, max_width=30):
    widths = np.arange(min_width, max_width + 1)
    peak_indices = find_peaks_cwt(intensity, widths)
    return [{"q": float(q[i]), "intensity": float(intensity[i])} for i in peak_indices]
```

5. scipy.signal.argrelextrema / argrelmax / argrelmin

Functionality: finds relative extrema using an order window and comparator. argrelmax is for local maxima; argrelmin is for local minima.

Best use case: low-level extrema detection when you want custom filtering, turning-point detection, baseline valley detection, or local minima between neighboring peaks.

Snippet:

```python
import numpy as np
from scipy.signal import argrelmax, argrelmin


def local_extrema(q, intensity, order=5):
    y = np.asarray(intensity, dtype=float)
    maxima = argrelmax(y, order=order)[0]
    minima = argrelmin(y, order=order)[0]

    return {
        "maxima": [{"q": float(q[i]), "intensity": float(y[i])} for i in maxima],
        "minima": [{"q": float(q[i]), "intensity": float(y[i])} for i in minima],
    }
```

6. Recommended Scattering Workflow

Workflow:

1. Integrate detector image to I(q) or I(two-theta), if needed.
2. Remove NaN/inf values.
3. Smooth lightly with Savitzky-Golay or another conservative filter.
4. Estimate/adjust baseline.
5. Run find_peaks with prominence and distance.
6. Compute peak_prominences and peak_widths.
7. Fit selected peaks with Gaussian/Lorentzian/pseudo-Voigt model if quantitative parameters are needed.

Combined snippet:

```python
import numpy as np
from scipy.signal import find_peaks, peak_prominences, peak_widths, savgol_filter


def analyze_1d_scattering_curve(q, intensity):
    y = np.asarray(intensity, dtype=float)
    y = np.nan_to_num(y, nan=np.nanmedian(y), posinf=np.nanmax(y), neginf=0.0)
    y = savgol_filter(y, window_length=11, polyorder=3)
    y = y - np.percentile(y, 10)

    peaks, _ = find_peaks(y, prominence=0.04 * np.max(y), distance=8)
    prominences, left_bases, right_bases = peak_prominences(y, peaks)
    widths, width_heights, left_ips, right_ips = peak_widths(y, peaks, rel_height=0.5)

    dx = float(np.mean(np.diff(q)))
    records = []
    for i, peak in enumerate(peaks):
        records.append({
```

```
            "q": float(q[peak]),
            "intensity": float(y[peak]),
            "prominence": float(prominences[i]),
            "fwhm_q": float(widths[i] * dx),
            "left_base_q": float(q[left_bases[i]]),
            "right_base_q": float(q[right_bases[i]]),
        })
    return records
```